

สรุปผลการทดสอบประสิทธิภาพ (Performance Summary) — PairEval

k6 performance suite | เป้าหมาย: <http://localhost:8000> (uvicorn + SQLite) | ข้อมูลจำลอง: 100 นักศึกษา / 2,200 pairs | 6 ต.ค. 2026

สรุปประเด็นสำคัญ 4 ข้อ

รองรับการใช้งานปกติได้ดี	ข้อจำกัดในการประมวลผลข้อมูลแบบกลุ่ม
0.00% Error ที่การจำลองผู้ใช้งาน 21 คน (1,286 requests) ผ่านเกณฑ์ 100% ไม่มี Error 5xx เกิดขึ้นเลย แสดงว่าโค้ดปัจจุบันใช้เป็นมาตรฐานสำหรับ CI ได้	**ระบบรับคำสั่งได้ < 2 req/s** เมื่อเพิ่มอัตราคำสั่งเรียกดูคะแนนเป็น 2 req/s พบว่าความเร็ว (p95) ของ /my-score ลดลงอย่างมากจนเกิด Timeout หากมีคิวสะสมมากเกินไป

ระบบล่มเมื่อรับโหลดไม่ไหว (ไม่มีการจำกัดคิว)	ประสิทธิภาพลดลงเมื่อทำงานต่อเนื่อง
0 : 447 พบ 447 คำสั่งที่ล้นคิว (Timeout/500) และ 0 คำสั่งที่ถูกปฏิเสธอย่างสุภาพ (429) ระบบไม่มี Rate limit ทำให้เมื่อโหลดเกินขีดจำกัด ผู้ใช้จะพบหน้าจอตกหรือระบบล่ม แทนที่จะได้รับข้อความแจ้งให้ "ลองใหม่อีกครั้ง"	4.2% Error Rate ในระยะยาว ในการทดสอบระยะยาว (Soak test) พบอัตราคำสั่งล้นคิว 4.2% บ่งชี้ว่าระบบอาจมีปัญหาทรัพยากรตกราคงหรือมีคิวงานสะสมเมื่อเปิดใช้งานต่อเนื่อง

ผลรวมทั้ง 5 สถานการณ์ (Scenarios)

การทดสอบ	รูปแบบโหลดที่ใช้	จำนวนคำสั่ง (Requests)	อัตราความล้มเหลว	Error 5xx/Timeout	ผ่านเกณฑ์	วัตถุประสงค์
smoke.js	1 ผู้ใช้, 1 รอบการทำงาน	119	0.00%	0	PASS	ยืนยันการทำงานพื้นฐานของ API
load.js	ค่อยๆ เพิ่มผู้ใช้งานถึง 20 คน	1,286	0.00%	0	PASS	ด้าน CI สำหรับตรวจสอบคุณภาพโค้ด
soak.js	แช่โหลดผู้ใช้งานหนึ่งไว้ 3 นาที	423	4.25%	18	FAIL	หาปัญหาเมื่อระบบทำงานต่อเนื่อง
stress.js	เพิ่มผู้ใช้งานอย่างหนักจนถึง 120 คน	2,494	1.84%	46	FAIL	ดูพฤติกรรมระบบเมื่อรับโหลดเกินกำลัง
breakpoint.js	ค่อยๆ เพิ่มอัตราคำสั่ง	552	80.9%	447	FAIL	หาขีดจำกัดสูงสุดของระบบ (Capacity)

	ไปจนถึง 5 req/s					
--	-----------------	--	--	--	--	--

หมายเหตุ: *stress* และ *breakpoint* ถูกออกแบบมาให้ทำระบบล่มเพื่อสังเกตพฤติกรรม ดังนั้นการขึ้น *FAIL* จึงเป็นเรื่องปกติสำหรับการทดสอบนี้

รายละเอียดด้านทดสอบ CI (load.js ที่ 20 ผู้ใช้)

Endpoint	ระดับความสำคัญ	จบเวลา p95 (ms)	จำนวน (n)	p50 (ms)	p95 (ms)	p99 (ms)	ผลลัพธ์
GET /export/csv	งานเบื้องหลัง	30,000	0	0.0	0.0	0.0	PASS
GET /reports/individual	การแสดงผลหน้าจอ	3,000	0	0.0	0.0	0.0	PASS
GET /reports/coverage	การแสดงผลหน้าจอ	3,000	0	0.0	0.0	0.0	PASS
GET /reports/group	การแสดงผลหน้าจอ	3,000	0	0.0	0.0	0.0	PASS
GET /assignments/{id}/evaluations	การแสดงผลหน้าจอ	3,000	57	12.6	15.7	37.6	PASS
GET /my-score	งานเบื้องหลัง	30,000	29	24.7	70.3	88.7	PASS
POST ::recompute	งานเบื้องหลัง	30,000	1	34.1	34.1	34.1	PASS
POST /evaluations:submit	การแสดงผลหน้าจอ	3,000	57	3.0	3.2	59.1	PASS
POST /evaluations/draft (บันทึกอัตโนมัติ)	ตอบสนองทันที	200	1199	2.5	37.0	100.3	PASS

POST /auth/google	ตอบ สนอง ทันที	200	57	8.8	32.3	44.8	PASS
GET /notifications	ตอบ สนอง ทันที	200	143	43.9	45.9	56.5	PASS

สมมติฐาน เทียบกับ ผลลัพธ์จริง (Hypothesis vs Actual)

สมมติฐานที่ตั้งไว้	ผลลัพธ์จริงจากการทดสอบ	สรุป
GET /my-score จะเป็น API ที่ทำงานช้าที่สุด เพราะต้องคำนวณคะแนนใหม่ทั้งคลาสทุกครั้งที่ยเรียก (คาดว่า p95 > 500 ms)	p95 = 70.3 ms (ที่ระดับ 20 ผู้ใช้)	ผิดคาด (ในระดับโหลดทั่วไป)
GET /export/csv จะเป็น API ที่ช้าที่สุดของระบบโดยรวม	(ไม่มีข้อมูลการเรียกในรอบล่าสุด)	-
ระบบบันทึกผลอัตโนมัติ POST /evaluations/draft จะใช้เวลาได้อยู่ในงบ 200 ms แม้มีผู้ใช้งาน 20 คน	p95 = 37.0 ms จากคำสั่งทั้งหมด 1,199 คำสั่ง	ถูกต้อง
ขีดจำกัดของระบบถูกกำหนดโดยงานคำนวณสถิติรวม (Aggregate) ไม่ใช่จากปริมาณคำสั่งบันทึกข้อมูลย่อยๆ	จุดคอขวดอยู่ที่ < 2 req/s สำหรับงานประมวลผลรวม ในขณะที่คำสั่งบันทึกอัตโนมัติทำงานผ่านได้อย่างสบาย	ถูกต้อง

ทำข้อ 1 จึงผิดพลาด: ในการทดสอบโหลดปกติ (20 ผู้ใช้งาน) การคำนวณผล 2,200 คู่ประเมินยังสามารถทำงานเสร็จในเวลาเพียง ~70 ms อย่างไรก็ตาม การทดสอบการทำลายขีดจำกัด (breakpoint test) แสดงให้เห็นว่า API เดียวกันนี้ใช้เวลาประมวลผลนานถึง 120 วินาทีจนระบบตัดการเชื่อมต่อ (Timeout) หากมีการเรียกใช้งานเกิน 2 คำสั่งต่อวินาที นี่เป็นการยืนยันว่าการใช้การทดสอบหลายแบบมีความจำเป็น

จุดคอขวดของระบบที่ตรวจพบ (System Bottlenecks)

1. ระบบคำนวณคะแนนของทั้งคลาสใหม่ทุกครั้งที่ถูกเรียกดู

ข้อมูลจากโค้ด: ฟังก์ชัน `run_scoring_engine_for_assignment()` มีการดึงข้อมูลการจับคู่และผลประเมินของทั้งคลาสขึ้นมาคำนวณใหม่เสมอ โดยไม่มีการทำ Cache หรือเก็บบันทึกผลลัพธ์ลงตาราง **ผลกระทบ:** การะงานต่อ 1 คำสั่งประมวลผล จะแปรผันตามจำนวนคู่ประเมินทั้งหมดในคลาสนั้นๆ หมายความว่าหากนักศึกษาค 100 คนเปิดดูคะแนนตัวเองพร้อมกัน ระบบจะต้องคำนวณข้อมูลของทั้งคลาสใหม่ซ้ำถึง 100 รอบ แทนที่จะคำนวณเฉพาะข้อมูลของบุคคลนั้น

2. ไม่มีระบบปฏิเสธคำขอเมื่อโหลดเกินกำลัง (No Load Shedding)

ไม่มีการจำกัดอัตราการเรียก (Rate limit) และไม่มีการกำหนดความยาวคิวงาน ส่งผลให้เมื่อมีคำขอมากเกินไปที่เซิร์ฟเวอร์จะรับได้ คำขอเหล่านั้นจะค้างอยู่ในระบบจนถึงเวลา Timeout ผู้ใช้งานจะพบอาการหน้าจอก้างหรือ Error 500 (ระบบขัดข้อง) แทนที่จะได้รับการตอบสนองด้วยสถานะ 429 Too Many Requests เพื่อแจ้งเตือนให้ผู้ใช้งาน "ลองใหม่อีกครั้ง" ในภายหลัง การไม่มีระบบปฏิเสธที่ดีทำให้การกู้คืนระบบทำได้ยากขึ้นและให้ประสบการณ์ผู้ใช้ที่ไม่ดี

3. คอขวดจากการเขียนฐานข้อมูล (Database Locking)

ในการทดสอบ stress test พบว่า POST /evaluations:submit มีค่าเวลาตอบสนองโดยเฉลี่ย 3.2 วินาที แต่ค่า p99 กลับพุ่งสูงถึง 59.1 วินาที (ห่างกันเกือบ 20 เท่า) ตัวเลขที่กระโดดห่างกันเช่นนี้คือรูปแบบที่ชัดเจนของปัญหา Database Locking จากการใช้ SQLite ไฟล์เดียว อย่างไรก็ตาม ข้อนี้อาจเป็นเพียงข้อจำกัดของสภาพแวดล้อมทดสอบ จำเป็นต้องทดสอบบน PostgreSQL ก่อนสรุปผล

แนวทางการแก้ไขในอนาคต

ลำดับ	แนวทาง	ผลลัพธ์ที่คาดหวัง
1	เก็บบันทึกผลลัพธ์การคำนวณลงฐานข้อมูล (Persist) และอัปเดตใหม่เมื่อจำเป็น เพื่อให้ GET /my-score ดึงข้อมูลที่คำนวณเสร็จแล้วไปใช้งานได้ทันที	ภาระงานจะลดลงจากระดับรวมทั้งคลาสเหลือเพียง $O(1)$ ซึ่งน่าจะทำให้ p95 ลดลงเหลือระดับ $< 30\text{ ms}$ ได้
2	ปรับปรุง get_student_score_view ให้คำนวณเฉพาะคะแนนของผู้ที่เรียกดู แทนที่จะคำนวณผลรวมของทั้งคลาส	ลดภาระงานต่อ Request ลงประมาณ 100 เท่า สำหรับคลาสเรียนขนาด 100 คน
3	เพิ่มระบบ Rate Limit และ Timeout บนฝั่งแอปพลิเคชันให้ตอบสถานะ 429 เมื่อคิวเต็ม	ผู้ใช้จะไม่เจออาการค้าง และระบบจะปฏิเสธคำร้องด้วยความสุภาพ ช่วยป้องกันไม่ให้เซิร์ฟเวอร์ล่มโดยรวม
4	เปลี่ยนฐานข้อมูลทดสอบหลักเป็น PostgreSQL ให้ตรงกับ Production	แก้ปัญหความแตกต่างของ Database Locking ที่เกิดขึ้นจาก SQLite